

Role: Expert Deep Learning TA & Interactive Visualization Engineer ^

Task: Create a single-file, self-contained HTML/JS application (using Tailwind CSS via CDN) that serves as an interactive "Multi-Head Attention (MHA) Dimension Tracer."

Target Audience: Graduate students working through a PyTorch implementation of the Transformer's Multi-Head Attention layer who need to understand the tensor shape manipulation (the `split_heads` and `combine_heads` functions).

Pedagogical Goal (Addressing Question 3):

Visually trace the tensor's shape through the MHA forward pass, specifically demonstrating the effect of the following functions on a sample input tensor with a defined batch size, sequence length, and d_{model} :

Linear Layer: $(B, L, d_{\text{model}}) \rightarrow (B, L, d_{\text{model}})$

Split Heads: $(B, L, d_{\text{model}}) \rightarrow (B, H, L, d_k)$

Scaled Dot Product: $(B, H, L, d_k) \rightarrow (B, H, L, d_k)$ (Output of $\text{softmax}(QK^T / \sqrt{d_k})V$)

Combine Heads: $(B, H, L, d_k) \rightarrow (B, L, d_{\text{model}})$

Final Linear Layer: $(B, L, d_{\text{model}}) \rightarrow (B, L, d_{\text{model}})$

Key Features & Requirements:

Input Definition:

Hardcode and display the following values: Batch Size ($B=2$), Sequence Length ($L=4$), d_{model} ($D=12$), Number of Heads ($H=3$).

Automatically calculate and display: d_k ($D_k = D/H = 4$).

Interactive Shape Tracer:

Provide a vertical list of the key tensor operations in the forward function (corresponding to the PyTorch code provided in the original prompt).

Next to each operation, display a block showing the tensor's shape at that point, e.g., (2, 4, 12).

Focus on the `split_heads` and `combine_heads` steps. Use visual aids (like color-coding or a simple block diagram) to show which dimensions were created/merged/transposed.

Question 3 (a)-(h) Interactive Blanks:

Present the PyTorch code snippet (Question 3) directly in a dedicated card.

Use clickable/fillable blanks for (a) through (h).

When the user clicks a blank, provide a detailed textual explanation of why that value is correct, linking it back to the dimension tracing visualization (e.g., for (g) in `split_heads`, explain that the sequence length must be preserved and the D_{model} dimension is being reshaped into $H \times D_k$).

Add score multiplied with V , and V multiplied with W_o

Results

1. Input Tensor

Previous Next Operation ➡



Start with input X (Shape: 4x12). This contains the embeddings for 4 tokens.

2. Split Heads (Visual Slice)

Previous Next Operation ➡



We logically slice the vectors into 3 parts (Red, Green, Blue).

3. Split Heads (Physical Separate)

Previous

Next Operation



The slices are pulled apart into 3 separate Heads (Shape: 3 x 4x4).

4. Prepare Attention (Head 1)

Previous

Next Operation



Focusing on the Red Head: We have Query (Q) and Key (K) matrices.

5. Transpose Key (K)

[Previous](#)[Next Operation](#)

Query (Q)



Key^T

To multiply, we rotate K to become K^T (Shape: 4x4 -> 4x4).

6. Attention Scores (Q @ K^T)

[Previous](#)[Next Operation](#)

K^T



Q



Scores (Softmax)



Dot Product: Row of Q * Col of K^T = Score. Highlighted: Q_{row_0} attending to K_{col_0}.

7. Weighted Sum (Scores @ V)

[Previous](#)[Next Operation](#)

We multiply the Attention Scores (probabilities) by the Value matrix (V). This creates the Context Vector for this head.

8. Combine Heads

[Previous](#)[Next Operation](#)

We perform this for all 3 heads, then concatenate the results back together.

9. Final Linear Layer (Context @ W_o)

[Previous](#)[Next Operation](#)

Finally, the combined matrix (4x12) is multiplied by the Output Weight Matrix W_o (12x12) to mix information across heads.